Algorithmique

et programmation 3

Complexité · Récursivité · Tri · Liste Chaînées · Arbre Binaires

Saïd Jabbour
UFR des Sciences · Licence Sciences et Technologie · Semestre 3
Année universitaire 2026–2027

Plan du cours

1. Complexité
2. Récursivité
3. La dichotomie
4. Algorithmes de Tri
5. Algorithmes de Tri
6. Listes chaînées
7. Une version itérative pour les listes chaînées

Évaluation

- Deux contrôles continus : CC1, CC2
- Travaux pratiques : TP
- Un projet / SAE : PRJ

Fil conducteur

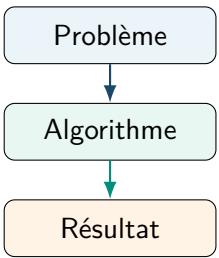
1. Résoudre un problème
2. Choisir une structure adaptée
3. Écrire l'algorithme
4. Analyser sa complexité
5. Comparer plusieurs solutions

Formule donnée dans les modalités du cours

$$CC = \frac{2\,CC1 + 2\,CC2 + TP}{5}, \qquad \text{Note Algo3} = CC, \qquad \text{Note SAE Algo3} = PRJ.$$

Pourquoi étudier les algorithmes ?

- Certains problèmes sont simples à résoudre, mais peuvent être mal résolus en termes de coût.
- D'autres deviennent rapidement difficiles lorsque la taille des données augmente.
- Deux programmes **corrects** peuvent avoir des performances très différentes.



Quelques problèmes algorithmiques

Problèmes classiques

- Maximum / minimum d'une liste
- Deuxième plus grande valeur
- Tri d'une liste
- Recherche d'une valeur

Problèmes plus riches

- Problème des n reines
- Parcours du cavalier
- Fouille de texte
- Coloriage / couverture de graphes
- Recherche de motifs fréquents

À retenir

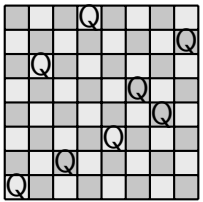
La difficulté ne vient pas seulement d'obtenir une réponse : il faut chercher une méthode qui reste exploitable quand n augmente.

Exemple : le problème des huit reines

Objectif

Placer 8 reines sur un échiquier 8×8 sans que deux reines puissent se capturer.

- Une contrainte sur chaque ligne
- Une contrainte sur chaque colonne
- Une contrainte sur les diagonales

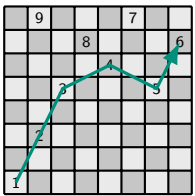


Exemple : le parcours du cavalier

Problème

Trouver un parcours du cavalier qui visite les cases de l'échiquier selon les règles du mouvement du cavalier.

- Recherche avec retour arrière (backtracking)
- Choix de l'ordre d'exploration
- Importance de l'espace de recherche



Exemple : fouille de motifs ensemblistes

Client	Achats
1	œufs, farine, levure, sucre
2	œufs, farine, fromage, yaourt
3	œufs, farine, course, boisson
4	pâtes, yaourt, sauce tomate
5	pâtes, sauce tomate

Support au moins égal à 2

$$\{\text{œufs}, \text{farine}\} \qquad \text{et} \qquad \{\text{pâtes}, \text{sauce tomate}\}$$

À retenir

La formulation du problème et la structure des données influencent directement les algorithmes possibles.

Exemple : résoudre un système d'inéquations

Système

$$
\begin{aligned}
x_1 + x_2 &\geq 1 \\
x_1 - x_2 &\geq 0 \\
-x_1 + x_2 &\geq 0 \\
-x_1 - x_2 &\geq -1
\end{aligned}
$$

Domaine

$$x_1, x_2 \in \{0, 1\}.$$

On ne cherche donc que parmi quatre affectations possibles.

À vous de réfléchir

Quelles affectations satisfont simultanément les quatre contraintes ?

Complexité

Pourquoi mesurer la complexité ?



- Temps d'exécution
- Mémoire utilisée
- Comparaison lorsque la taille des données augmente

Comparer deux algorithmes

Même problème : calculer le minimum et le maximum

Version 1 : une seule boucle

```
def min_max_v1(liste):
    mini, maxi = liste[0], liste[0]
    for x in liste[1:]:
        if x > maxi:
            maxi = x
        elif x < mini:
            mini = x
    return mini, maxi
```

Version 2 : deux boucles

```
def min_max_v2(liste):
    mini, maxi = liste[0], liste[0]
    for x in liste[1:]:
        if x < mini:
            mini = x
    for x in liste[1:]:
        if x > maxi:
            maxi = x
    return mini, maxi
```

À vous de réfléchir

Les deux algorithmes mettent le même temps ?

Mesurer un coût : l'idée des opérations élémentaires

Complexité en temps

$C(A, D)$ = temps d'exécution de l'algorithme A appliqué aux données D.

Eléments de calcul de la complexité en temps

- C(opération élémentaire) = constant
- C(si F alors I sinon J) $C(F) + max(C(I), C(J))$
- C(pourtout i de e_1 à e_2 faire E_i) $\leq C(e_1) + C(e_2) + \sum C(E_i)$
- Temps de calcul de procédures récursives : solution d'équations de récurrence

Comparaisons de complexités : Ordres de grandeur asymptotique

- $C(n) = O(f(n))$

  $\exists k > 0$ et $N \in \mathbb{N}$ tels que $\forall n \geq N$ on a $C(n) \leq k\ f(n)$.
  On dit alors que $C(n)$ est "grand O" de $f(n)$.

- $C(n) = \Omega(f(n))$

  $\exists k > 0$ et $N \in \mathbb{N}$, tels que $\forall n \geq N$, $C(n) \geq kf(n)$
  On dit alors que $C(n)$ est "grand oméga" de $f(n)$.

- $C(n) = \Theta(f(n))$

  si $C(n) = O(f(n))$ et $C(n) = \Omega(f(n))$
  On dit alors que $C(n)$ est "grand thêta" de $f(n)$.

Mesurer un coût : l'idée des opérations élémentaires

Principe

On ne mesure généralement pas les secondes exactes : on compte un nombre d'opérations élémentaires en fonction de n.

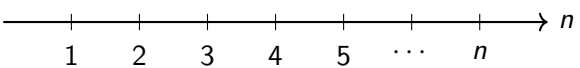
- Une opération élémentaire est considérée de coût constant : $\Theta(1)$.
- Une séquence additionne les coûts.
- Une boucle répète le coût de son corps.
- Une condition dépend du chemin exécuté ; pour le pire cas, on considère le chemin le plus coûteux.
- Une fonction récursive conduit souvent à une **équation de récurrence**.

À retenir

La complexité est une **fonction de la taille des données**, pas une mesure absolue du temps sur une machine particulière.

Exemple : compter une boucle

```
for i in range(n):
    traitement_constant()
```



Conclusion

Le corps est exécuté n fois : $T(n) = an + b$, donc $T(n) = \Theta(n)$.

Pire cas, meilleur cas, cas moyen

Soit $C(A, D)$ le coût de l'algorithme A sur les données D.

$$C_{\max}(A, n) = \max_{|D|=n} C(A, D),$$
$$C_{\min}(A, n) = \min_{|D|=n} C(A, D),$$
$$C_{\text{moy}}(A, n) = \sum_{|D|=n} p(D)\, C(A, D).$$

Relation générale

$$C_{\min}(A, n) \leq C_{\text{moy}}(A, n) \leq C_{\max}(A, n).$$

Le cas moyen nécessite de préciser le modèle probabiliste $p(D)$: sans hypothèse sur la distribution des entrées, il n'est pas défini.

Ordres de grandeur

n	$\log_2 n$	$\sqrt{n}$	$n \log_2 n$	n^2	n^3	2^n
5	2.32	2.24	11.6	25	125	32
10	3.32	3.16	33.2	100	1 000	1 024
100	6.64	10	664	10 000	10^6	$\approx 10^{30}$
1 000	9.97	31.62	9 970	10^6	10^9	$\approx 10^{301}$
10 000	13.29	100	132 877	10^8	10^{12}	$\approx 10^{3010}$

Message essentiel

Une différence apparemment faible pour petit n peut devenir énorme lorsque n augmente.

Comparer des fonctions : $\mathcal{O}$, Ω et Θ

$\mathcal{O}$: borne supérieure asymptotique

$$C(n) = \mathcal{O}(f(n)) \iff \exists k > 0, \exists N, \ \forall n \geq N : \ C(n) \leq kf(n).$$

Ω : borne inférieure asymptotique

$$C(n) = \Omega(f(n)) \iff \exists k > 0, \exists N, \ \forall n \geq N : \ C(n) \geq kf(n).$$

Θ : même ordre de grandeur

$$C(n) = \Theta(f(n)) \iff C(n) = \mathcal{O}(f(n)) \text{ et } C(n) = \Omega(f(n)).$$

Règles pratiques de simplification

- $3n + 7 = \Theta(n)$: on conserve le terme dominant.
- $n^2 + 100n + 50 = \Theta(n^2)$.
- $5 \log n + 20 = \Theta(\log n)$.
- $n + n^2 = \Theta(n^2)$.
- Deux boucles *successives* donnent souvent une somme de coûts.
- Deux boucles *imbriquées* donnent souvent un produit de coûts.

À retenir

On ne supprime pas arbitrairement les termes : on justifie pourquoi ils deviennent négligeables asymptotiquement.

Exercice guidé : calcul de complexité

Algorithme 1

```
def maximum(liste):
    res = liste[0]
    for x in liste[1:]:
        if x > res:
            res = x
    return res
```

À vous de réfléchir

Exprimer le nombre d'itérations en fonction de n.

Algorithme 2

```
def somme(n):
    s = 0
    for i in range(n-1):
        for j in range(i+1, n):
            s += i + j
    return s
```

À vous de réfléchir

Combien de fois l'instruction est-elle exécutée ?

Correction : exercice de complexité

Algorithme 1

La boucle parcourt $n-1$ éléments :

$$T_1(n) = an + b = \Theta(n).$$

Algorithme 2

Pour un i fixé, j prend $n-i-1$ valeurs. Donc

$$\sum_{i=0}^{n-2}(n-i-1) = 1+2+\cdots+(n-1) = \frac{n(n-1)}{2}.$$

Ainsi $T_2(n) = \Theta(n^2)$.

Exercice guidé : calcul de complexité

Algorithme 1

```
def algo3 (n):
    while n >1:
        print (n)
        n = n // 2
```

À vous de réfléchir

Exprimer le nombre d'itérations en fonction de n.

Algorithme 2

```
def algo3 (n):
    m = 1
    while m < n:
        print (n)
        m = m * 2
```

À vous de réfléchir

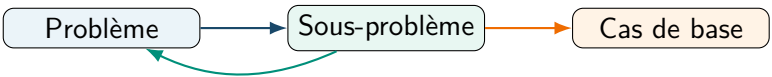
Combien de fois l'instruction est-elle exécutée ?

Récursivité

Le principe de récursivité

Définition

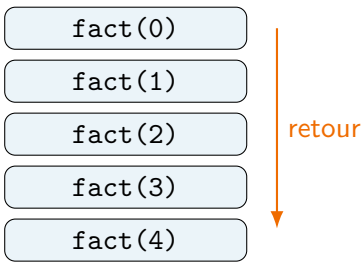
Une fonction est **récursive** lorsqu'elle s'appelle elle-même sur une instance plus petite ou plus proche d'un cas d'arrêt.



- **Cas de base** : arrête la récursion.
- **Cas récursif** : transforme le problème en un problème plus petit.
- Sans progrès vers le cas de base, la récursion ne termine pas.

La pile d'appels

Pour un appel comme `fact(4)`, les appels sont empilés :



À retenir

Une fonction récursive utilise la **pile d'exécution**. La profondeur de récursion peut donc devenir une contrainte mémoire.

Récursivité terminale vs non terminale

Terminale

L'appel récursif est la **dernière opération** effectuée.

```
f(n) = f(n-1)
```

Non terminale

Une opération reste à effectuer après le retour de l'appel.

```
f(n) = 1 + f(n-1)
```

À retenir

Python n'effectue pas d'optimisation générale des appels terminaux. Une récursion terminale n'évite donc pas, en Python, l'empilement des appels.

Exemple : factorielle

Version itérative

```
def fact_iter(n):
    res = 1
    for i in range(2, n + 1):
        res *= i
    return res
```

$T(n) = \Theta(n)$, espace auxiliaire $\Theta(1)$

Version récursive

```
def fact_rec(n):
    if n == 0:
        return 1
    return n * fact_rec(n - 1)
```

$T(n) = \Theta(n)$, profondeur $\Theta(n)$

À vous de réfléchir

Même complexité temporelle, mais même coût mémoire ?

Dérouler une récursion

Exemple : $\mathrm{fact}(4)$

$$\begin{aligned}\mathrm{fact}(4) &= 4 \times \mathrm{fact}(3)\\ &= 4 \times 3 \times \mathrm{fact}(2)\\ &= 4 \times 3 \times 2 \times \mathrm{fact}(1)\\ &= 4 \times 3 \times 2 \times 1 \times \mathrm{fact}(0)\\ &= 24.\end{aligned}$$

- Phase 1 : descente jusqu'au cas de base.
- Phase 2 : remontée et calcul des résultats.

Exercice : nombre d'occurrences

Objectif

Écrire une fonction récursive qui compte le nombre d'occurrences d'un caractère dans une chaîne.

```
def nb_occurrences(car, chaine):
    if len(chaine) == 0:
        return 0
    if car == chaine[0]:
        return 1 + nb_occurrences(car, chaine[1:])
    return nb_occurrences(car, chaine[1:])
```

À vous de réfléchir

Identifier le cas de base, le cas récursif et la taille du sous-problème.

Point d'attention : les tranches Python

Dans l'exemple précédent, `chaine[1:]` crée une nouvelle chaîne.

Conséquence

Une analyse trop rapide pourrait conclure à $\Theta(n)$ en comptant seulement les appels récursifs. Avec les chaînes Python, la création répétée des tranches peut ajouter un coût non négligeable.

Version plus adaptée à l'analyse

Passer un indice i plutôt que créer une nouvelle chaîne à chaque appel :

```
nb_occurrences(car, chaine, i)
```

À retenir

Pour analyser un algorithme, il faut tenir compte des **opérations réellement effectuées par le langage**.

Itérative versus récursive

- La plupart des traitements itératifs simples sont facilement traduisibles sous forme récursive (exemple du for)
- L'inverse est faux
- Il arrive même qu'un problème ait une solution récursive triviale alors qu'il est très difficile d'en trouver une solution itérative

Définition récursive de la fonction d'Ackermann

$$Ack(m,n) = \begin{cases} n+1, & \text{si } m = 0, \\ Ack(m-1,1), & \text{si } m > 0 \text{ et } n = 0, \\ Ack\big(m-1, Ack(m,n-1)\big), & \text{si } m > 0 \text{ et } n > 0. \end{cases}$$

Itérative versus récursive

- $u_0 = 1$
- $u_n = 2u_{n-1} + 3$

Itérative versus récursive

```
def terme(n):
    if n == 0:
        return 1
    return 2 * terme(n - 1) + 3

def terme(n,acc=1):
  if n==0:
    return acc
  return terme(n-1, 2*acc+3)

  def terme_iter(n):
  val=1
  for i in range (1,n+1):
    val = 2*val+3
  return val
```

Fonctions récursives - À vous !

Exercice

Donner une version itérative, récursive non terminale et récursive terminale de la suite de fibonacci définit comme suit :

- $fibo(0) = 1$
- $fibo(1) = 1$
- $fibo(n) = fibo(n-1) + fibo(n-2),\ n \geq 2$

Fonctions récursives - À vous !

```python
# 1. Récursivité simple
def fibo_rec(n:int) -> int:
    if n <= 1:
        return 1
    return fibo_rec(n-1) + fibo_rec(n-2)

# 2. Version itérative
def fibo_iter(n:int) -> int:
    p, q = 0, 1
    for _ in range(n):
        p, q = q, p + q
    return q

# 3. Récursivité terminale
def fibo_term(n:int, p=0, q=1) -> int:
    if n == 0:
        return q
    return fibo_term(n-1, q, p+q)
```

Itérative versus récursive

..

La dichotomie

Recherche : linéaire ou dichotomique ?

Liste non triée

On ne peut pas éliminer arbitrairement des éléments.

Recherche linéaire : jusqu'à n comparaisons.

Liste triée

L'ordre permet d'éliminer la moitié de la zone de recherche à chaque étape.

Recherche dichotomique : $\Theta(\log n)$.



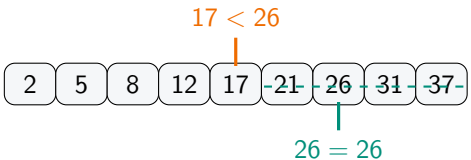
Principe de la recherche dichotomique

1. Considérer une zone de recherche triée.
2. Examiner l'élément du milieu.
3. Si la valeur recherchée est plus petite, garder la moitié gauche.
4. Si elle est plus grande, garder la moitié droite.
5. Répéter jusqu'à trouver la valeur ou obtenir une zone vide.

Précondition essentielle

La liste doit être triée selon le même ordre que celui utilisé par les comparaisons.

Une trace pas à pas



Recherche de 26 dans $[2, 5, 8, 12, 17, 21, 26, 31, 37]$.

Idée clé

En quelques comparaisons, on élimine de grandes portions de la liste.

Implémentation itérative

```
def recherche(l, val):
    """Retourne un indice de val dans l, ou -1 si val est absente.
    Précondition : l est triée dans l'ordre croissant.
    """
    deb = 0
    fin = len(l)                  # intervalle [deb, fin[
    while deb < fin:
        milieu = (deb + fin) // 2
        if l[milieu] == val:
            return milieu
        elif l[milieu] < val:
            deb = milieu + 1
        else:
            fin = milieu
    return -1
```

À retenir

Implémentation récursive

```python
def recherche_rec(l, val, deb, fin):
    """Recherche val dans l[deb:fin].
    Précondition : l est triée dans l'ordre croissant.
    """
    if deb >= fin:
        return -1

    milieu = (deb + fin) // 2
    if l[milieu] == val:
        return milieu
    if l[milieu] < val:
        return recherche_rec(l, val, milieu + 1, fin)
    return recherche_rec(l, val, deb, milieu)
```

Appel initial : `recherche_rec(l, val, 0, len(l))`.

Pourquoi $\log_2 n$?

Après k étapes, la taille maximale de la zone restante est approximativement

$$\frac{n}{2^k}.$$

On s'arrête lorsque cette quantité devient inférieure à 1 :

$$\frac{n}{2^k} \leq 1 \quad \Longleftrightarrow \quad 2^k \geq n \quad \Longleftrightarrow \quad k \geq \log_2 n.$$

Conclusion

La recherche dichotomique effectue $\Theta(\log n)$ comparaisons dans le pire cas.

Recherche linéaire vs dichotomie

Taille n	Linéaire : au plus n comparaisons	Dichotomie : ordre de grandeur
4	4	2
10	10	4
50	50	6
10 000	10 000	14
100 000	100 000	17

Attention aux conditions de comparaison

La dichotomie est très efficace, mais elle suppose une donnée **triée**. Si les données changent souvent, le coût du tri doit aussi être pris en compte.

Synthèse : les idées à maîtriser

Complexité

- taille n
- coût dominant
- $\mathcal{O}$, Ω, Θ
- pire / moyen / meilleur cas

Récursivité

- cas de base
- sous-problème
- pile d'appels
- récurrence

Dichotomie

- liste triée
- division par 2
- invariants de bornes
- $\Theta(\log n)$

À retenir

Réflexe d'algorithmicien : **problème → hypothèses → algorithme → preuve de correction → complexité → comparaison**.

Exercices pour aller plus loin

1. Écrire une recherche dichotomique qui retourne le premier indice d'une valeur répétée.
2. Donner une version récursive de la recherche du maximum d'une liste.
3. Calculer la complexité de trois boucles imbriquées et comparer avec une version récursive équivalente.
4. Modifier `nb_occurrences` pour éviter les tranches et analyser sa complexité temporelle et spatiale.
5. Expliquer pourquoi une liste non triée ne permet pas directement d'utiliser la dichotomie.

Questions ?

Algorithmes de Tri

Algorithmes de Tri

Tri : Objectif

- **Au départ :** Éléments dans un ordre quelconque.

5	2	12	3	5	...

- **À la fin :** Les **mêmes** éléments ordonnés (croissant ou décroissant).

2	3	5	5	12	...

Principe général des tris

Idée fondamentale

Réaliser des **permutations (échanges)** d'éléments pour passer de la situation initiale à la situation triée.

Objectif d'efficacité

Effectuer le **moins de comparaisons et de permutations** possible.

Tri par sélection : Principe

Principe du tri par sélection

1. On cherche le plus petit élément du tableau non trié.
2. On l'échange avec l'élément situé au début de la zone non triée.
3. On répète l'opération sur le reste du tableau.

À vous de réfléchir

Écrire les deux fonctions nécessaires :

- La recherche de l'indice du minimum à partir d'un indice donné.
- La boucle principale du tri par sélection.

Tri par sélection : Recherche du minimum

```
def recherche_mini(l, ind):
    '''
    Retourne l'indice du plus petit element de l
    situe entre l'indice ind et la fin de la liste.
    '''
    assert ind < len(l)
    mini = ind
    for i in range(ind, len(l)):
        if l[i] < l[mini]:
            mini = i
    return mini
```

Tri par sélection : Implémentation

```
def tri_selection(l):
    '''
    Trie la liste l en place par selection du minimum.
    '''
    for i in range(len(l)):
        # Recherche de l'indice du plus petit element
        ind_mini = recherche_mini(l, i)
        # Permutation avec l'element a la position i
        l[i], l[ind_mini] = l[ind_mini], l[i]
```

Tri par sélection : Complexité

Soit n le nombre d'éléments de la liste.
Pour chaque position i (de 0 à $n-1$) :

- On parcourt les $n-i$ éléments restants.
- On effectue $n-i$ comparaisons.
- On réalise 1 permutation.

Calcul du nombre de comparaisons

$$\sum_{i=0}^{n-1}(n-i) = \frac{n(n+1)}{2} = \frac{n^2+n}{2}$$

À retenir

Le tri par sélection nécessite toujours $\Theta(n^2)$ comparaisons, indépendamment de l'état initial du tableau. Sa complexité temporelle est **quadratique :** $\mathcal{O}(n^2)$.

Tri par sélection : Amélioration (Min-Max)

Idée d'optimisation

À chaque étape, chercher simultanément le **minimum** et le **maximum** pour placer deux éléments à leur position définitive.

```
def recherche_min_max(l, deb, fin):
    assert deb < fin
    mini, maxi = deb, fin - 1
    for i in range(deb, fin):
        if l[i] < l[mini]:
            mini = i
        if l[i] > l[maxi]:
            maxi = i
    return mini, maxi
```

Tri à bulles : Principe

Principe du tri à bulles

On parcourt le tableau en comparant les éléments adjacents. S'ils sont dans le mauvais ordre, on les permute.

- À chaque passage, le plus grand élément **« remonte »** comme une bulle vers sa position finale à la fin du tableau.

À retenir

Complexité dans le pire des cas : $\mathcal{O}(n^2)$.

Tri à bulles : Implémentation optimisée

```python
def tri_a_bulles(l):
    '''
    Tri a bulles avec arret premature si la liste est triee.
    '''
    bulle = True
    i = 0
    while i < len(l) and bulle:
        bulle = False
        for ind in range(len(l) - i - 1):
            if l[ind] > l[ind + 1]:
                # Permutation
                l[ind], l[ind + 1] = l[ind + 1], l[ind]
                bulle = True
        i += 1
```

Tri par insertion : Principe

Principe du tri par insertion

Semblable à la façon dont on trie des cartes en main :

1. On prend les éléments un par un à partir du deuxième.
2. On insère l'élément à sa juste place dans la sous-liste déjà triée à sa gauche, en décalant les éléments plus grands.

À retenir

- Pire des cas (liste inversée) : $\mathcal{O}(n^2)$.
- Meilleur des cas (liste déjà triée) : $\mathcal{O}(n)$.

Tri par insertion : Implémentation

```
def tri_insertion(l):
    '''
    Trie la liste l par l'algorithme du tri par insertion.
    '''
    for i in range(1, len(l)):
        ref = l[i]
        j = i - 1
        # Decalage des elements plus grands que ref
        while j >= 0 and l[j] > ref:
            l[j + 1] = l[j]
            j -= 1
        l[j + 1] = ref
```

Tri Fusion : Diviser pour régner

Principe récursif

1. **Diviser :** couper la liste en deux moitiés.
2. **Régner :** trier récursivement les deux sous-listes.
3. **Combiner :** fusionner les deux sous-listes triées en une seule liste triée.

À retenir

La fusion de deux sous-listes triées de taille globale n se fait en temps linéaire $\mathcal{O}(n)$.

Tri Fusion : Algorithme de fusion

def triFusion(a) :

```
def triFusion(a) :
  #renvoi  la liste des elements de a tries
  n = len(a)
  if n <= 1:
    return a
  m = n // 2
  return fusionner(triFusion(a[0:m]),triFusion(a[m:n]))
```

fusionner deux listes triées

```
def fusionner(a,b) :
  c = []
  n,m = len(a),len(b)
  i,j = 0,0
  while i < len(a) and j < len(b):
     if a[i] < b[j] :
       c.append(a[i])
       i = i + 1
     else :
       c.append(b[j])
       j = j + 1
  if i == len(a):
    for k in range(j,len(b)):
      c.append(b[k])
  else:
    for k in range(i,len(a)):
      c.append(a[k])
  return c
```

Tri Fusion : Implémentation récursive (Version 2)

```
def tri_fusion(l, deb, fin):
    '''
    Trie le troncon l[deb:fin] par tri fusion.
    '''
    if fin - deb >= 2:
        milieu = (deb + fin) // 2
        tri_fusion(l, deb, milieu)
        tri_fusion(l, milieu, fin)
        fusionner(l, deb, milieu, fin)
```

fusion (Version 2)

```
def fusionner(l, deb, milieu, fin):
    part1, part2 = l[deb:milieu], l[milieu:fin]
    i, j, k = 0, 0, deb

    while i < len(part1) and j < len(part2):
        if part1[i] < part2[j]:
            l[k] = part1[i]
            i += 1
        else:
            l[k] = part2[j]
            j += 1
        k += 1

    while i < len(part1):
        l[k] = part1[i]
        i, k = i + 1, k + 1
    while j < len(part2):
        l[k] = part2[j]
```

Tri Fusion : Complexité Bilan

Équation de récurrence

Le nombre d'opérations $C(n)$ pour un tableau de taille n vérifie :

$$C(n) = 2 \cdot C\left(\frac{n}{2}\right) + \mathcal{O}(n)$$

Complexité du tri fusion

$$C(n) = \Theta(n \log_2(n))$$

Avantages et inconvénients

- **Avantage :** Optimal sur le plan temporel ($\mathcal{O}(n \log n)$ dans tous les cas).
- **Inconvénient :** Nécessite une mémoire auxiliaire proportionnelle à n (complexité spatiale $\mathcal{O}(n)$).

Tri Rapide (QuickSort)

Principe (C.A.R. Hoare, 1962)

Aussi basé sur l'approche **Diviser pour régner** :

1. Choisir un élément appelé **pivot**.
2. **Partitionner** le tableau : placer les éléments inférieurs au pivot à sa gauche, et les supérieurs à sa droite.
3. Trier récursivement les deux sous-tableaux (sans étape de fusion !).

Tri Rapide : Partitionnement

```
def partitionner(l, deb, fin):
    pivot = l[fin - 1]
    borne_inf = deb
    borne_sup = fin - 1

    while borne_inf < borne_sup:
        if l[borne_inf] < pivot:
            borne_inf += 1
        else:
            l[borne_sup] = l[borne_inf]
            l[borne_sup - 1] = l[borne_sup - 1]
            l[borne_sup - 1] = l[borne_inf]
            borne_sup -= 1

    l[borne_inf] = pivot
    return borne_inf
```

Tri Rapide : Implémentation

```
def tri_rapide(l, deb, fin):
    '''
    Trie la liste l en place par l'algorithme du tri rapide.
    '''
    if deb < fin - 1:
        ind_pivot = partitionner(l, deb, fin)
        tri_rapide(l, deb, ind_pivot)
        tri_rapide(l, ind_pivot + 1, fin)
```

Tri Rapide : Analyse de complexité

La complexité dépend fortement du **choix du pivot** :

- **Pire des cas :** Pivot toujours le plus petit ou le plus grand (tableau déjà trié par exemple) $\longrightarrow \Theta(n^2)$.
- **Cas moyen :** Les partitions sont équilibrées $\longrightarrow \Theta(n \log_2(n))$.

À retenir

En pratique, le tri rapide est très efficace car il s'effectue **en place** (pas de tableau secondaire) et possède de petites constantes cachées.

Synthèse des algorithmes de tri

Algorithme	Meilleur cas	Cas moyen	Pire cas	Mémoire
Tri Sélection	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	$\mathcal{O}(1)$
Tri à Bulles	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$\mathcal{O}(1)$
Tri Insertion	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$\mathcal{O}(1)$
Tri Fusion	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$\mathcal{O}(n)$
Tri Rapide	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n^2)$	$\mathcal{O}(\log n)$

Visualisation interactive

Pour comparer visuellement l'exécution de ces algorithmes :
`https://www.sorting-algorithms.com/`